Tutorial: Pipeline de dados NASA DONKI/SEP

Grupo 6 · Diogo e Brenno · GitHub Actions + Supabase + GitHub Pages

1. O que este projeto faz

O projeto coleta automaticamente eventos de **partículas solares energéticas (SEP)** da API aberta DONKI da NASA, guarda os dados em um banco PostgreSQL no Supabase e os exibe em um painel web publicado no GitHub Pages. Este guia permite que uma pessoa externa replique o projeto completo.

Fluxo:

1. O **GitHub Actions** executa o script `scripts/fetch_nasa.py` (manualmente ou por agendamento diário).
2. O script consulta a API da NASA, normaliza os campos, remove duplicados e envia os dados ao **Supabase** em lotes (upsert).
3. Cada execução é registrada na tabela `execucoes`.
4. O `index.html`, no **GitHub Pages**, lê os dados pela API REST do Supabase e mostra tabela, filtros e a data da última atualização.

2. A API: eventos SEP

SEP são aumentos de partículas de alta energia (principalmente prótons) vindas do Sol, associados a erupções solares e ejeções de massa coronal. Podem afetar satélites, comunicações e astronautas. A DONKI (*Space Weather Database Of Notifications, Knowledge, Information*) registra esses eventos com data, instrumentos que os detectaram e vínculos com outros eventos.

Endpoint:
`https://api.nasa.gov/DONKI/SEP?startDate=AAAA-MM-DD&endDate=AAAA-MM-DD&api_key=SUA_CHAVE`

Campos usados: `sepID` (identificador único, chave natural), `eventTime`, `submissionTime`, `versionId`, `instruments`, `link` e `linkedEvents`.

3. Pré-requisitos e estrutura final

- Conta no GitHub e conta no Supabase (plano gratuito).
- Chave gratuita da NASA em api.nasa.gov (*Generate API Key*), enviada por e-mail.
- Navegador. Nada precisa ser instalado: tudo pode ser feito pelos sites.

Estrutura final do repositório:

```
DONKI-SEP/
├── index.html
├── README.md
├── scripts/fetch_nasa.py
├── sql/setup.sql
├── .github/workflows/update-data.yml
└── docs/ (tutorial.pdf, ai-interaction.md, reflexao.md, apresentacao.pdf)
```

4. Passo 1: testar a API da NASA

Antes de escrever código, cole no navegador:

```
https://api.nasa.gov/DONKI/SEP?startDate=2026-10-01&endDate=2026-10-31&api_key=SUA_CHAVE
```

Se aparecer uma lista JSON (mesmo vazia, `[]`), a chave e o endpoint funcionam. **Importante:** a API recusa intervalos muito longos (erro 400), por isso o script consulta em janelas de 30 dias.

5. Passo 2: organização e repositório no GitHub

1. Crie a organização `SN-2026-GRUPO-06-NASA`.
2. Dentro dela, crie o repositório **público** `DONKI-SEP`.
3. Convide o professor (`GilFerques`) em *People* → *Invite member*.
4. Ative o Pages em *Settings* → *Pages*: *Deploy from a branch*, branch `main`, pasta `/ (root)`.

Para criar pastas pelo site: *Add file* → *Create new file* e digite o caminho completo no campo do nome (por exemplo `sql/setup.sql`). A barra `/` cria a pasta automaticamente.

6. Passo 3: projeto no Supabase

1. Crie a organização `SN-2026-GRUPO-06-NASA`.
2. Crie o projeto `DONKI-SEP-DB` na região **South America (São Paulo)**.
3. Convide `rafael.ferques@ifpr.edu.br` como **Developer** em *Settings* → *Team*.

Criar as tabelas

No **SQL Editor**, abra uma *New query*, cole o conteúdo de `sql/setup.sql` e clique em **Run**. A mensagem “Success. No rows returned” é normal.

```
create table if not exists public.sep_events (
  id bigserial primary key,
  sep_id text not null,
  event_time timestamptz not null,
  submission_time timestamptz,
  version_id int,
  instruments text[],
  link text,
  linked_events jsonb,
  raw jsonb,
  inserted_at timestamptz default now(),
  constraint uq_sep_events_sep_id unique (sep_id)  -- chave natural (permite upsert)
);

create table if not exists public.execucoes (
  id bigserial primary key,
  executado_em timestamptz default now(),
  registros_processados int default 0,
  lotes int default 0,
  erros int default 0,
  status text check (status in ('concluido','erro_parcial','erro_critico')),
  mensagem text
);

alter table public.sep_events enable row level security;
alter table public.execucoes  enable row level security;

create policy "leitura publica sep_events" on public.sep_events for select using (true);
create policy "leitura publica execucoes"  on public.execucoes  for select using (true);

grant usage on schema public to anon, authenticated, service_role;
grant select on public.sep_events, public.execucoes to anon, authenticated;
grant all on public.sep_events, public.execucoes to service_role;
grant usage, select on all sequences in schema public to service_role;
```

O que cada parte faz

- `unique (sep_id)`: chave natural do evento; impede duplicatas e permite o upsert.
- **Tabela** `execucoes`: log de cada rodada (registros, lotes, erros, status).
- **RLS + políticas**: com RLS ligado, ninguém acessa sem política; as políticas liberam só leitura pública.
- **GRANTs**: `anon` e `authenticated` só leem; apenas `service_role` (pipeline) escreve.

Confira no **Table Editor**: devem aparecer `sep_events` e `execucoes`, vazias.

7. Passo 4: script de coleta

Crie `scripts/fetch_nasa.py`:

```
import os, sys, requests
from datetime import date, timedelta

SUPABASE_URL = os.environ["SUPABASE_URL"].rstrip("/")
SERVICE_KEY = os.environ["SUPABASE_SERVICE_KEY"]
NASA_KEY = os.environ.get("NASA_API_KEY", "DEMO_KEY")
DAYS = int(os.environ.get("DAYS", "365"))
BATCH = 100

HDR = {"apikey": SERVICE_KEY, "Authorization": f"Bearer {SERVICE_KEY}",
       "Content-Type": "application/json"}

def log_execucao(proc, lotes, erros, status, msg=""):
    requests.post(f"{SUPABASE_URL}/rest/v1/execucoes", headers={**HDR, "Prefer": "return=minimal"},
        json={"registros_processados": proc, "lotes": lotes, "erros": erros,
              "status": status, "mensagem": msg[:500]}, timeout=30)

def fetch():
    fim = date.today()
    ini = fim - timedelta(days=DAYS)
    eventos = []
    atual = ini
    while atual < fim:
        fim_janela = min(atual + timedelta(days=29), fim)
        r = requests.get("https://api.nasa.gov/DONKI/SEP",
            params={"startDate": atual.isoformat(), "endDate": fim_janela.isoformat(),
                    "api_key": NASA_KEY}, timeout=60)
        if not r.ok:
            raise RuntimeError(f"{r.status_code}: {r.text[:300]}")
        eventos.extend(r.json() or [])
        atual = fim_janela + timedelta(days=1)
    return eventos

def normalize(e):
    return {
        "sep_id": e.get("sepID"),
        "event_time": e.get("eventTime"),
        "submission_time": e.get("submissionTime"),
        "version_id": e.get("versionId"),
        "instruments": [i.get("displayName") for i in (e.get("instruments") or [])],
        "link": e.get("link"),
        "linked_events": e.get("linkedEvents"),
        "raw": e,
    }

def main():
    try:
        rows = [normalize(e) for e in fetch()]
    except Exception as ex:
        log_execucao(0, 0, 1, "erro_critico", str(ex)); print("ERRO:", ex); sys.exit(1)

    unicos = {r["sep_id"]: r for r in rows if r["sep_id"]}  # dedup (evita erro 21000)
    rows = list(unicos.values())
    print(f"{len(rows)} eventos únicos")

    lotes = erros = enviados = 0
    for i in range(0, len(rows), BATCH):
        lote = rows[i:i+BATCH]
        resp = requests.post(f"{SUPABASE_URL}/rest/v1/sep_events?on_conflict=sep_id",
            headers={**HDR, "Prefer": "resolution=merge-duplicates,return=minimal"},
            json=lote, timeout=60)
        lotes += 1
        if resp.ok: enviados += len(lote)
        else: erros += 1; print("Lote falhou:", resp.status_code, resp.text)

    status = "concluido" if erros == 0 else "erro_parcial"
    log_execucao(enviados, lotes, erros, status)
    if erros: sys.exit(1)

if __name__ == "__main__":
    main()
```

Decisões importantes

- **Janelas de 30 dias**: a API recusa intervalos longos.
- **Deduplicação por** `sep_id` antes do envio: evita o erro PostgreSQL 21000 (mesmo registro duas vezes no mesmo upsert).
- **Upsert** (`on_conflict=sep_id` + `merge-duplicates`): rodar todo dia não duplica dados.
- **Falha visível**: se um lote falha, registra `erro_parcial` e termina com `sys.exit(1)`, deixando o Actions vermelho.

8. Passo 5: workflow do GitHub Actions

Crie `.github/workflows/update-data.yml` **na raiz** do repositório (a pasta `.github` não pode ficar dentro de `scripts/`).

```
name: Atualizar dados DONKI-SEP
on:
  schedule:
    - cron: "0 6 * * *"
  workflow_dispatch:
permissions:
  contents: read
concurrency:
  group: update-data
  cancel-in-progress: false
jobs:
  update:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install requests
      - run: python scripts/fetch_nasa.py
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_SERVICE_KEY: ${{ secrets.SUPABASE_SERVICE_KEY }}
          NASA_API_KEY: ${{ secrets.NASA_API_KEY }}
```

Boas práticas presentes: `permissions: contents: read`, `concurrency` e `timeout-minutes`. O `workflow_dispatch` permite rodar manualmente; o `schedule` roda todo dia às 6h UTC.

9. Passo 6: Secrets no GitHub

No **repositório** (não na organização): *Settings → Secrets and variables → Actions → New repository secret*.

Nome	Valor	Onde pegar
`SUPABASE_URL`	`https://SEU-PROJETO.supabase.co`	Supabase → Project Settings → API → Project URL
`SUPABASE_SERVICE_KEY`	secret key (service_role)	Supabase → Project Settings → API Keys
`NASA_API_KEY`	chave recebida por e-mail	api.nasa.gov

Atenção ao `SUPABASE_URL`**:** use somente o endereço do projeto. Se for copiado com `/rest/v1/` no final, o Supabase responde `404 PGRST125 Invalid path specified in request URL`.

10. Passo 7: executar o pipeline

1. Aba **Actions** → **Atualizar dados DONKI-SEP** → **Run workflow** → branch `main`.
2. Aguarde alguns segundos e atualize: o ícone deve ficar verde.
3. No Supabase, em **Table Editor**, confira linhas em `sep_events` e uma linha com status `concluido` em `execucoes`.

Se aparecer apenas “Get started with GitHub Actions”, o workflow não está em `.github/workflows/` na raiz da branch `main`.

11. Passo 8: painel (index.html)

Crie `index.html` na raiz. Troque `SUA_URL` e `SUA_PUBLISHABLE_KEY` pelos valores do projeto. **Somente a publishable key** pode ficar neste arquivo.

```
<!DOCTYPE html>
<html lang="pt-BR"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>DONKI/SEP — Partículas Solares Energéticas</title>
<style>
body{font-family:system-ui,sans-serif;max-width:960px;margin:2rem auto;padding:0
    1rem;background:#0b1020;color:#e8ecf5}
a{color:#7fb2ff} table{width:100%;border-collapse:collapse} th,td{padding:.5rem;border-bottom:1px solid
    #26304d;text-align:left}
select,input{padding:.4rem;margin:0 .5rem .5rem 0} .cards{display:flex;gap:1rem;margin:1rem 0}
.card{background:#151c33;padding:1rem;border-radius:8px;flex:1}
</style></head><body>
<h1>*□ Partículas Solares Energéticas (DONKI/SEP)</h1>
<p id="update">Carregando última atualização…</p>
<div class="cards"><div class="card">Eventos: <b id="total">—</b></div><div class="card">Exibidos: <b
    id="shown">—</b></div></div>
<label>Ano: <select id="ano"><option value="">Todos</option></select></label>
<label>Instrumento: <input id="inst" placeholder="ex: GOES"></label>
<table><thead><tr><th>ID</th><th>Data do evento</th><th>Instrumentos</th><th>Link</th></tr></thead><tbody
    id="rows"></tbody></table>
<script>
const SUPABASE_URL = "SUA_URL";
const ANON_KEY = "SUA_PUBLISHABLE_KEY";
const H = {apikey: ANON_KEY, Authorization: "Bearer " + ANON_KEY};
function escapeHtml(v){return String(v ??
    "").replace(/[&<>"']/g,c=>({"&":"&amp;","<":"&lt;",">":"&gt;",'"':"&quot;","'":"&#39;"}[c]));}
let data = [];
async function load(){
  const r = await fetch(`${SUPABASE_URL}/rest/v1/sep_events?select=sep_id,event_time,instruments,link&order=ev
      ent_time.desc&limit=1000`,{headers:H});
  data = await r.json();
  const anos = [...new Set(data.map(d=>d.event_time.slice(0,4)))].sort().reverse();
  document.getElementById("ano").innerHTML = '<option value="">Todos</option>' +
      anos.map(a=>`<option>${escapeHtml(a)}</option>`).join("");
  const u = await fetch(`${SUPABASE_URL}/rest/v1/execucoes?select=executado_em,status&order=executado_em.desc&
      limit=1`,{headers:H}).then(r=>r.json());
  document.getElementById("update").textContent = u[0] ? `Última atualização: ${new
      Date(u[0].executado_em).toLocaleString("pt-BR")} (${u[0].status})` : "Sem execuções registradas";
  document.getElementById("total").textContent = data.length;
  render();
}
function render(){
  const ano = document.getElementById("ano").value, inst =
      document.getElementById("inst").value.toLowerCase();
  const f = data.filter(d => (!ano || d.event_time.startsWith(ano)) && (!inst || (d.instruments||[]).join("
      ").toLowerCase().includes(inst)));
  document.getElementById("shown").textContent = f.length;
  document.getElementById("rows").innerHTML =
      f.map(d=>`<tr><td>${escapeHtml(d.sep_id)}</td><td>${escapeHtml(new
      Date(d.event_time).toLocaleString("pt-BR"))}</td><td>${escapeHtml((d.instruments||[]).join(",
      "))}</td><td>${d.link && /^https?:\/\//.test(d.link) ? `<a href="${escapeHtml(d.link)}" target="_blank"
      rel="noopener">ver</a>` : ""}</td></tr>`).join("");
}
document.getElementById("ano").onchange = render; document.getElementById("inst").oninput = render;
load().catch(e => document.getElementById("update").textContent = "Erro ao carregar dados.");
</script></body></html>
```

O painel tem filtro por ano e por instrumento, contadores, a última atualização lida de `execucoes` e a função `escapeHtml()` aplicada a todo valor externo inserido no HTML.

12. Passo 9: publicar no GitHub Pages

1. Confirme em *Settings* → *Pages*: *Deploy from a branch*, `main`, `/ (root)`.
2. Faça um **commit** no `index.html`. O Pages só publica depois de um commit novo; sem ele o link retorna 404.
3. Na aba **Actions**, aguarde `pages build and deployment` ficar verde.

4. Abra `https://sn-2026-grupo-06-nasa.github.io/DONKI-SEP/` e atualize com Ctrl + F5.

13. Passo 10: testar a API REST do Supabase

No navegador (troque pela publishable key):

```
https://SEU-PROJETO.supabase.co/rest/v1/sep_events?select=*&apikey=SUA_PUBLISHABLE_KEY
```

Deve aparecer um JSON com os eventos. Esse resultado é um dos itens da entrega.

14. Segurança das chaves

Chave	Pode ficar no index.html?	Onde usar
Publishable (`sb_publishable_...` / anon)	Sim: é pública e só lê, por causa do RLS	`index.html`
Secret (`sb_secret_...` / service_role)	**Nunca**	Somente no Secret `SUPABASE_SERVICE_KEY`
`NASA_API_KEY`	Não	Somente em Secret

Se uma chave secreta for exposta em código, commit, print ou mensagem, regenere-a imediatamente no Supabase e atualize o Secret no GitHub.

15. Erros comuns e soluções

Sintoma	Causa provável	Solução
Actions mostra “Get started”	Workflow fora de `.github/workflows` na raiz	Mover o arquivo na branch `main`
`400 Bad Request` na NASA	Intervalo de datas longo demais	Consultar em janelas de 30 dias
`403` da NASA	Chave inválida	Conferir o Secret `NASA_API_KEY`
`KeyError: 'SUPABASE_URL'`	Secret ausente ou com outro nome	Criar o Secret com o nome exato
`404 PGRST125` no Supabase	`SUPABASE_URL` com `/rest/v1/` no final	Usar só `https://...supabase.co`
`relation "sep_events" does not exist`	`setup.sql` não foi executado	Rodar o SQL no projeto correto
`401` / `permission denied`	Chave errada ou GRANTs ausentes	Usar a secret key no Secret e conferir os GRANTs
Painel dando 404	Pages sem publicação ou `index.html` fora da raiz	Fazer um commit e conferir o caminho
Painel “Erro ao carregar dados”	URL ou publishable key errada	Conferir as constantes e o Console (F12)
Workflow verde com 0 eventos	Período sem eventos SEP	Aumentar `DAYS`

16. Checklist da entrega

- Organização, repositório e projeto com os nomes da Tabela 02.
- Professor adicionado no GitHub e no Supabase (Developer).
- `sql/setup.sql` com UNIQUE, RLS e GRANTs.
- Três Secrets configurados.
- Workflow verde com dados reais em `sep_events` e registros em `execucoes`.
- Painel publicado com filtro, última atualização e `escapeHtml()`.

- `docs/` com `tutorial.pdf`, `ai-interaction.md`, `reflexao.md` e `apresentacao.pdf`.
- Planilha de reserva com os links definitivos (colunas G e H).
- PDF único no Classroom: links, print do workflow verde, print do Table Editor e URL da API REST retornando JSON.